



Lecture - 10

Informed/Heuristic Searching (Greedy best-first search and Heuristic function)

Course Teacher

Prof. Dr. Md. Ashikur Rahman Khan

Informed/Heuristic/Intelligent Search Strategies

Uninformed search is a **blind search**.

Informed Search is **Greedy search**
Also called Heuristic or Intelligent Search

An informed search strategy is one that uses (additional) problem-specific knowledge beyond the definition of the problem itself (that helps direct search to more promising paths).

Informed/Heuristic/Intelligent Search Strategies

Informed search algorithm contains an array of knowledge such as

- (i) how far we are from the goal,
- (ii) path cost,
- (iii) how to reach to goal node, etc.

This knowledge help agents to

- explore less to the search space and
- find more efficiently the goal node.

- ❑ The informed search algorithm is more useful for large search space.
- ❑ An informed search can find solutions more efficiently than can an uninformed strategy.

Comparison between Uninformed and Informed Search

	Uninformed Searching	Informed Searching
1	Search without information	Search with information
2	No knowledge	Use knowledge to find steps to solution
3	Time consuming	Quick solution
4	More time complexity	Less time complexity or reduces time
5	More space complexity	Less space complexity
6.	Obtain optimal solution	May or may not obtain optimal solution
7.	BFS (Goes to all nodes), DFS etc	A*, Heuristic DFS, Best first search (Does not go to all nodes)
8.	It gives exponential number of nodes	It gives non polynomial (NP) nodes number.

Comparison between Uninformed and Informed Search

- Heuristic search **performance better** as compared with blind search in terms of time.
- Heuristic is a technique designed to **solve a problem quickly**. That's why we use Heuristic search when we want quick solution.
- The heuristic method, however, **might not always give the best solution**, but **it guaranteed to find a good solution in reasonable time**.
- It calculates the cost of an optimal path between the pair of states.

Heuristics function

A heuristic function, $h(n)$, provides an estimate of the cost of the path from a given node to the closest goal state. And it must be zero if node represents a goal state.

It is represented by $h(n)$.

The value of the heuristic function **is always positive**.

Heuristic function is the most common form in which **additional knowledge** of the problem **is imparted to the search algorithm**.

Example: Straight-line distance from current location to the goal location in a road navigation problem.

Informed (Heuristic) Search Strategies

Informed search may use

Euclidian distance or

Manhattan distance or

Number of misplaces.



Euclidian distance

$$= \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Informed (Heuristic) Search Strategies

The general approach is called best-first search.

Best-first search is an example of the general TREE-SEARCH or GRAPH-SEARCH algorithm in which a node is selected for expansion based on an evaluation function, $f(n)$.

The evaluation function is construed as a cost estimate, so the node with the lowest evaluation is expanded first.

The implementation of best-first graph search is identical to that for uniform-cost search (Figure 3.14), except for the use of f instead of g to order the **priority queue**.

In Best-first search, a node is selected for expansion based on an evaluation function, $f(n)$.

Informed (Heuristic) Search Strategies

The choice of f determines the search strategy.

Most best-first algorithms include as a component of f a **heuristic function, denoted $h(n)$** :

$h(n)$ = estimated cost of the cheapest path from the state at node n to a goal state

= Straight Line Distance

= Euclidian Distance

Pure Heuristics Search

Pure Heuristic Search:

Pure heuristic search is the simplest form of heuristic search algorithms.

It expands nodes based on their heuristic value $h(n)$.

It maintains two lists, OPEN and CLOSED list.

In the CLOSED list, it places those nodes which have already expanded and in the OPEN list, it places nodes which have yet not been expanded.

On each iteration, each node n with the lowest heuristic value is expanded and generates all its successors and n is placed to the closed list.

The algorithm continues until a goal state is found.

Types of Heuristic Search

The types of Informed/Heuristic Search:

1. Best First Search
2. A*search
3. Local Beam Search
4. Stochastic Beam Search

Greedy best-first search

Greedy best-first search tries to expand the node that is closest to the goal, on the grounds that this is likely to lead to a solution quickly.

Also called **Best-First search** or **Greedy search**

Thus, it evaluates nodes by using just the heuristic function; that is, $f(n)=h(n)$.

if n is a goal node, then $h(n)=0$.

Thus, it evaluates nodes by using just the heuristic function; that is,

$$f(n) = h(n).$$

Algorithm of Greedy best-first search

Algorithm:

Let 'OPEN' be a priority containing initial state.

Loop

 if OPEN is empty return failure

Node $\leftarrow$ Remove-First (OPEN)

 if Node is a Goal

 then return the path from initial to Node

 else generate all successors of Node and put the newly generated Node into OPEN according to their f values

END Loop

Algorithm of Greedy best-first search

Or

Step 1: Place the starting node into the OPEN list.

Step 2: If the OPEN list is empty, Stop and return failure.

Step 3: Remove the node n , from the OPEN list which has the lowest value of $h(n)$, and places it in the CLOSED list.

Step 4: Expand the node n , and generate the successors of node n .

Step 5: Check each successor of node n , and find whether any node is a goal node or not. If any successor node is goal node, then return success and terminate the search, else proceed to Step 6.

Step 6: For each successor node, algorithm checks for evaluation function $f(n)$, and then check if the node has been in either OPEN or CLOSED list. If the node has not been in both list, then add it to the OPEN list.

Step 7: Return to Step 2.

Algorithm of Greedy best-first search

Algorithm:

Let 'OPEN' be a priority containing initial state.

Loop

if OPEN is empty return failure

Node ← Remove-First (OPEN)

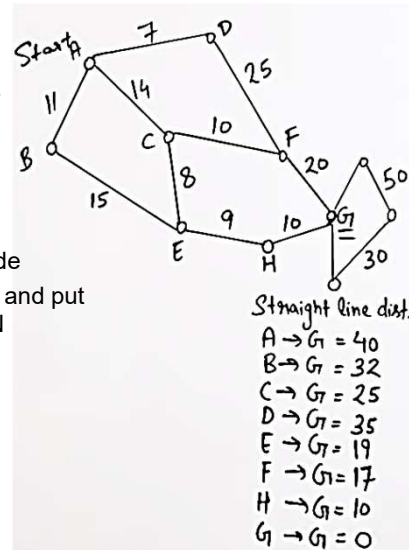
if Node is a Goal

then return the path from initial to Node

else generate all successors of Node and put the newly generated Node into OPEN according to their f values

END Loop

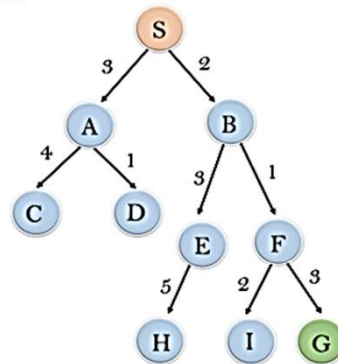
Best-First Search
concentrate on heuristic value of SLD.



Greedy best-first search (Contd.)

Example # 1:

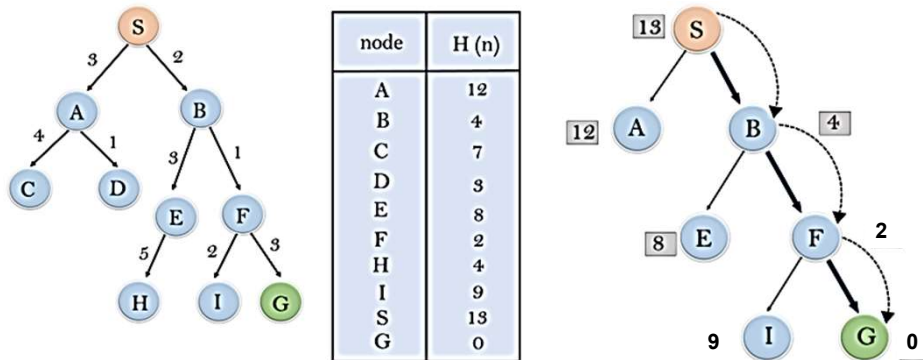
Consider the below search problem, and we will traverse it using greedy best-first search. At each iteration, each node is expanded using evaluation function $f(n)=h(n)$, which is given in the below table.



node	H (n)
A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
S	13
G	0

Example # 1 (Contd.)

In this search example, we are using two lists which are **OPEN** and **CLOSED** Lists. Following are the iteration for traversing the above example.



Example # 1 (Contd.)

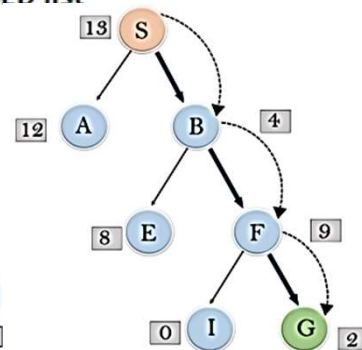
Expand the nodes of S and put in the CLOSED list

Initialization: Open [A, B], Closed [S]

Iteration 1: Open [A], Closed [S, B]

Iteration 2: Open [E, F, A], Closed [S, B]
: Open [E, A], Closed [S, B, F]

Iteration 3: Open [I, G, E, A], Closed [S, B, F]
: Open [I, E, A], Closed [S, B, F, G]



Hence the final solution path will be: **S----> B---->F----> G**

Example # 2:

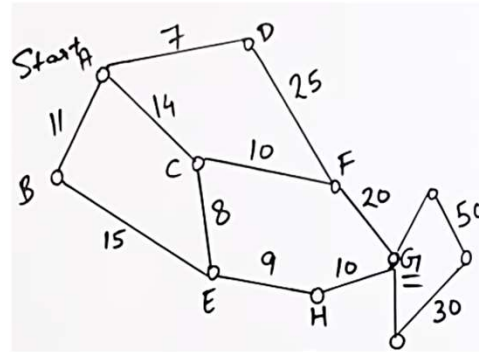
Here 11, 14, 7, etc are not the heuristic value. These are the cost to go to B, C, and D respectively from A.

Here f values is heuristic.

This heuristic is calculated by Euclidian Distance or Straight Line Distance.

The **straightline distance heuristic**, which is called h_{SLD} .

$$h_{SLD} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$



Straight line dist.

$$A \rightarrow G = 40$$

$$B \rightarrow G = 32$$

$$C \rightarrow G = 25$$

$$D \rightarrow G = 35$$

$$E \rightarrow G = 19$$

$$F \rightarrow G = 17$$

$$H \rightarrow G = 10$$

$$G \rightarrow G = 0$$

Example # 2 (Contd.):

Priority is given on **best heuristic value**.

Best heuristic value is this where the **cost is minimum**.

Straight line dist.

$$A \rightarrow G = 40$$

$$B \rightarrow G = 32$$

$$C \rightarrow G = 25$$

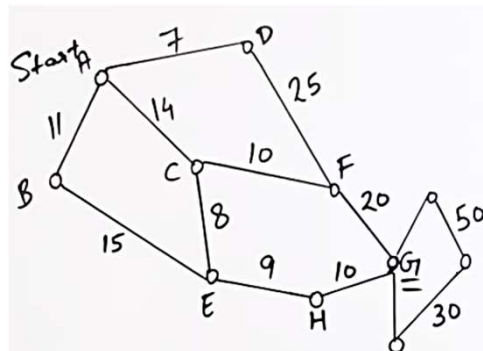
$$D \rightarrow G = 35$$

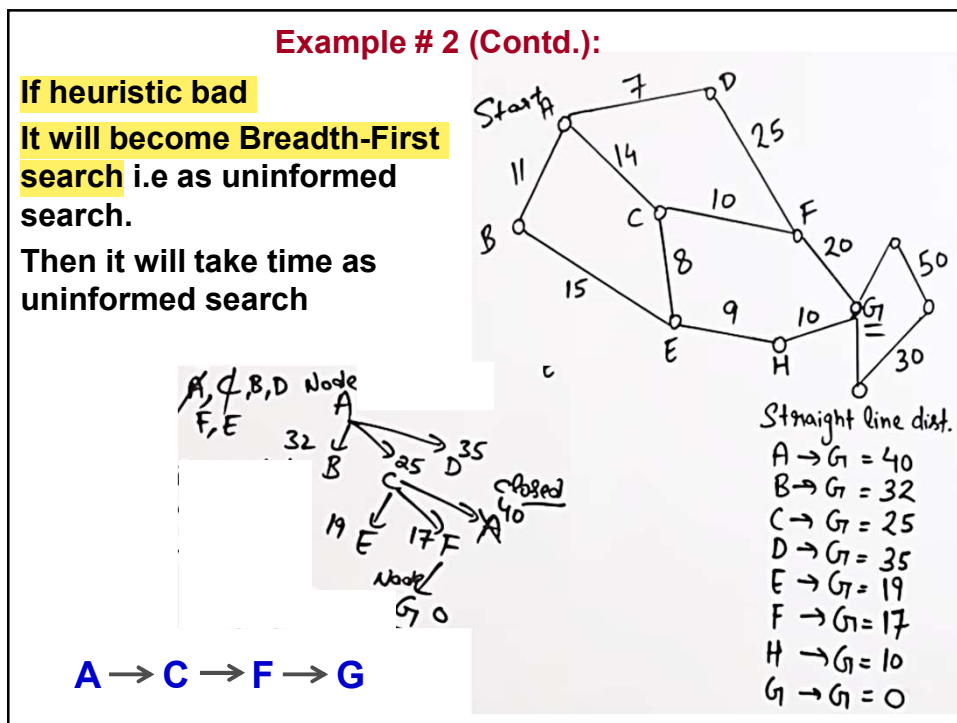
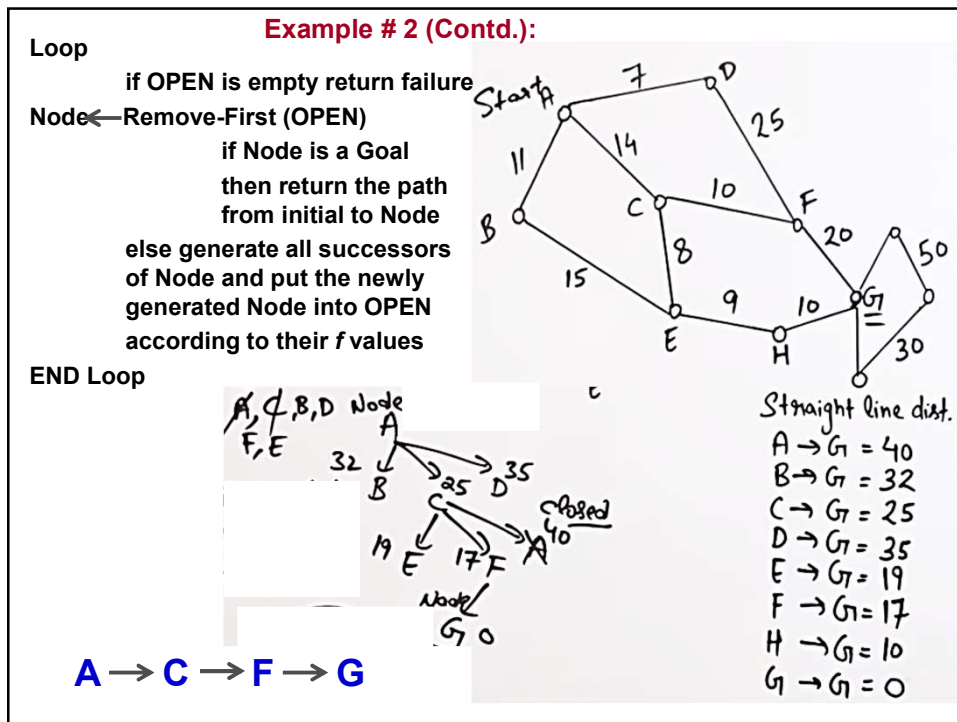
$$E \rightarrow G = 19$$

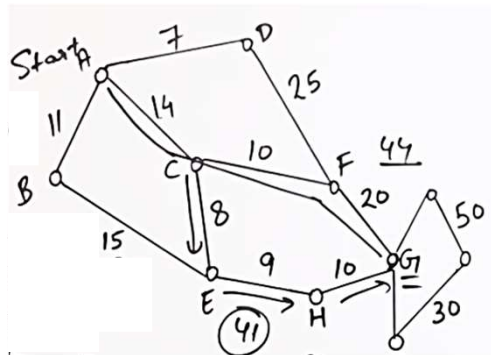
$$F \rightarrow G = 17$$

$$H \rightarrow G = 10$$

$$G \rightarrow G = 0$$





Example # 2 (Contd.):

Straight line dist.

$A \rightarrow G = 40$

$B \rightarrow G = 32$

$C \rightarrow G = 25$

$D \rightarrow G = 35$

$E \rightarrow G = 19$

$F \rightarrow G = 17$

$H \rightarrow G = 10$

$G \rightarrow G = 0$

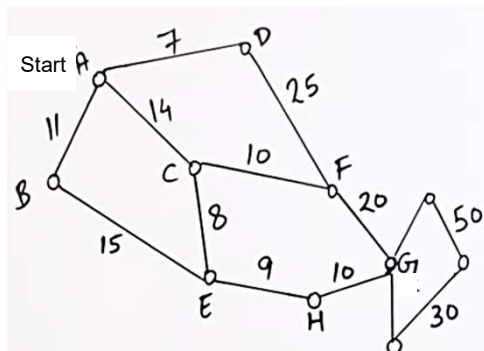
$A \rightarrow C \rightarrow F \rightarrow G = 44$

$A \rightarrow C \rightarrow E \rightarrow H \rightarrow G = 41$

**Write the Open
and Closed list
for this
problem??**

Example # 3:

In the following figure, 11, 7, 14, 25, 15, 8, 10, 9, 20, 10, 50 and 30 are the distances (or path cost) of the corresponding two states. The straight line distances are shown in following table at the right side. Find the solution (path) to reach the goal using greedy best-first search. Also find the path using breath-first search and consider that G is goal state.



Straight line distance

$A \rightarrow G = 40$

$B \rightarrow G = 32$

$C \rightarrow G = 25$

$D \rightarrow G = 35$

$E \rightarrow G = 19$

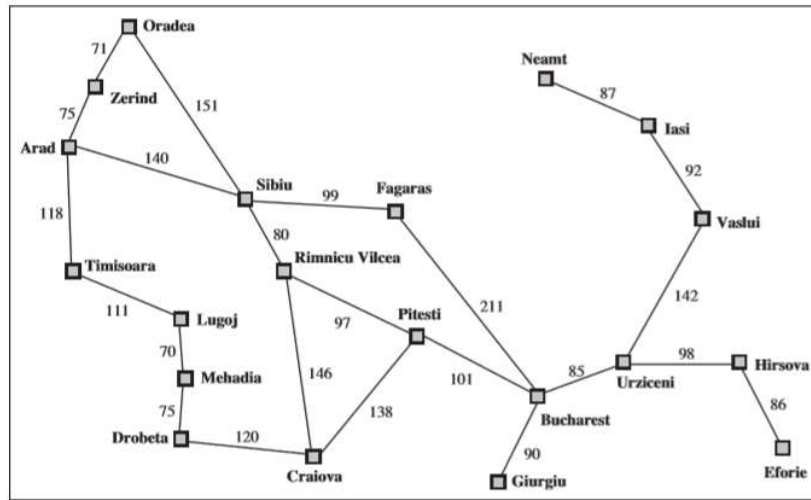
$F \rightarrow G = 17$

$H \rightarrow G = 10$

$G \rightarrow G = 0$

Example # 4:

In Romania, one might estimate the cost of the cheapest path from Arad to Bucharest via the straight-line distance from Arad to Bucharest.



Example # 4 (Contd.):

Let us see how this works for route-finding problems in Romania; we use the straight line distance heuristic, which we will call h_{SLD} .

If the goal is Bucharest, we need to know the straight-line distances to Bucharest, which are shown in Figure 3.22. For example, $h_{\text{SLD}}(\text{In}(\text{Arad}))=366$.

Notice that the values of h_{SLD} cannot be computed from the problem description itself.

Moreover, it takes a certain amount of experience to know that h_{SLD} is correlated with actual road distances and is, therefore, a useful heuristic.

Example # 4 (Contd.):

If the goal is Bucharest, we need to know the **straight-line distances** to Bucharest, which are shown in Figure 3.22.

Straight-line distances is also Euclidian distance

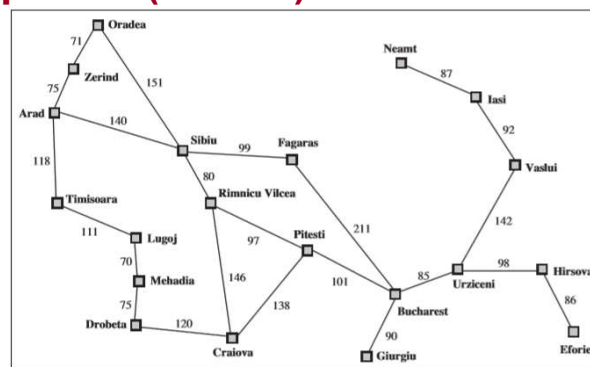
It can be calculated as

$$= \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Figure 3.22 Values of h_{SLD} —straight-line distances to Bucharest.

Example # 4 (Contd.):



Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Example # 4 (Contd.):

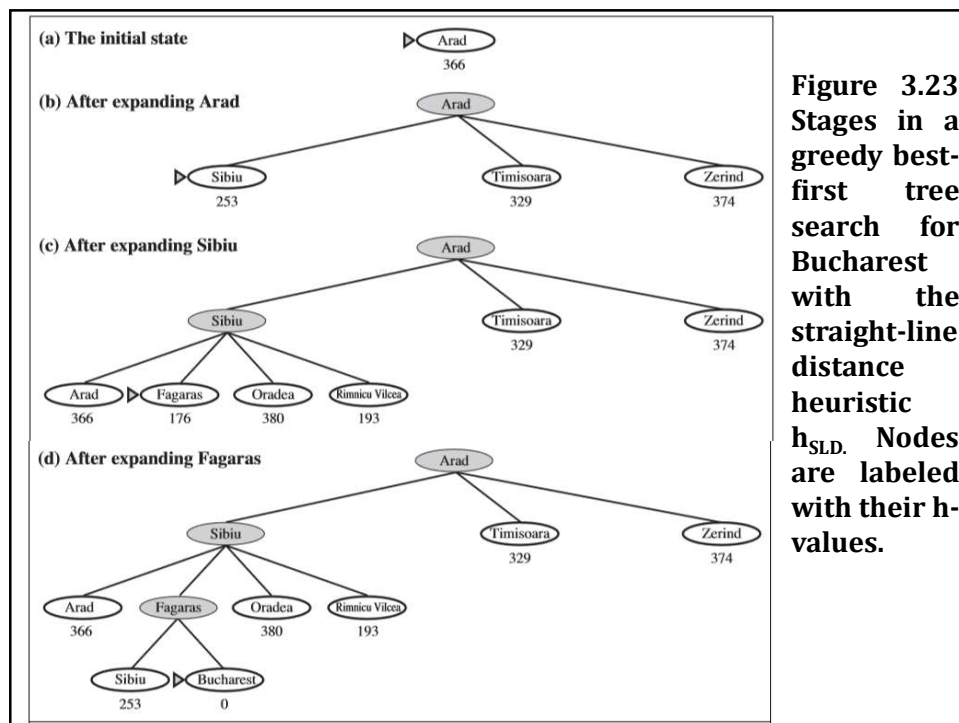
For example, $h_{SLD}(In(Arad))=366$.

Figure 3.23 shows the progress of a greedy best-first search using h_{SLD} to find a path from Arad to Bucharest.

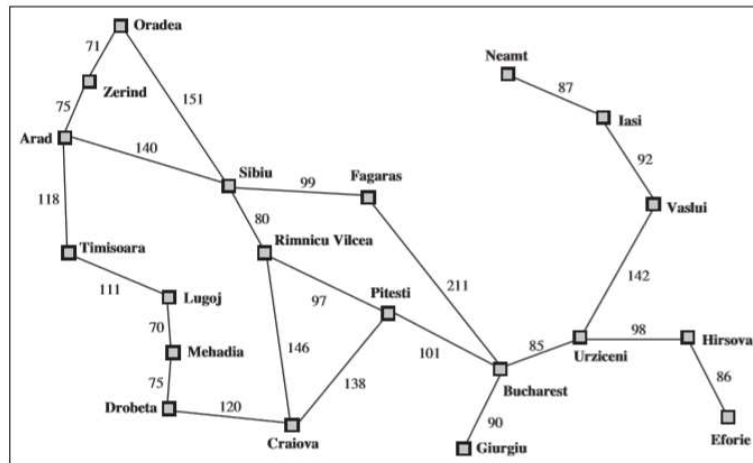
The first node to be expanded from Arad will be Sibiu because it is closer to Bucharest than either Zerind or Timisoara.

The next node to be expanded will be Fagaras because it is closest. Fagaras in turn generates Bucharest, which is the goal.

This shows **why the algorithm is called “greedy”**—**at each step it tries to get as close to the goal as it can.**



Example # 4 (Contd.):



Greedy best-first search (Contd.)

- **Time Complexity:** The worst case time complexity of Greedy best first search is $O(b^m)$.
- **Space Complexity:** The worst case space complexity of Greedy best first search is $O(b^m)$. Where, m is the maximum depth of the search space.
- **Complete:** Greedy best-first search is also incomplete, even if the given state space is finite.
- **Optimal:** Greedy best first search algorithm is not optimal.

Informed (Heuristic) Search Strategies

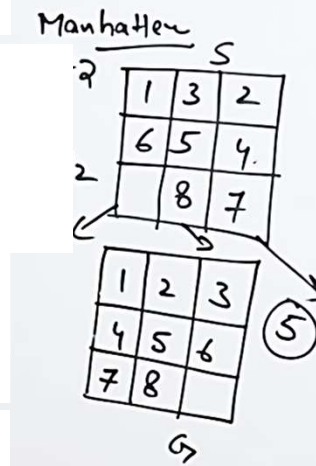
Informed search may use **Euclidian distance**

For 8-puzzle problem as below the **Manhattan distance** is

$$0+1+1+2+0+2+2+0 = 8$$

For 8-puzzle problem as below the **number of misplaced** is

$$1+1+1+1+1 = 5$$



8-Puzzle Problem with Heuristic (Greedy Best first search)

8- Puzzle Problem :

As

1	2	3
	4	6
7	5	8

S
(Start)

1	2	3
4	5	6
7	8	

G
(Goal)

Soln:

At start (S), what is its heuristic value?

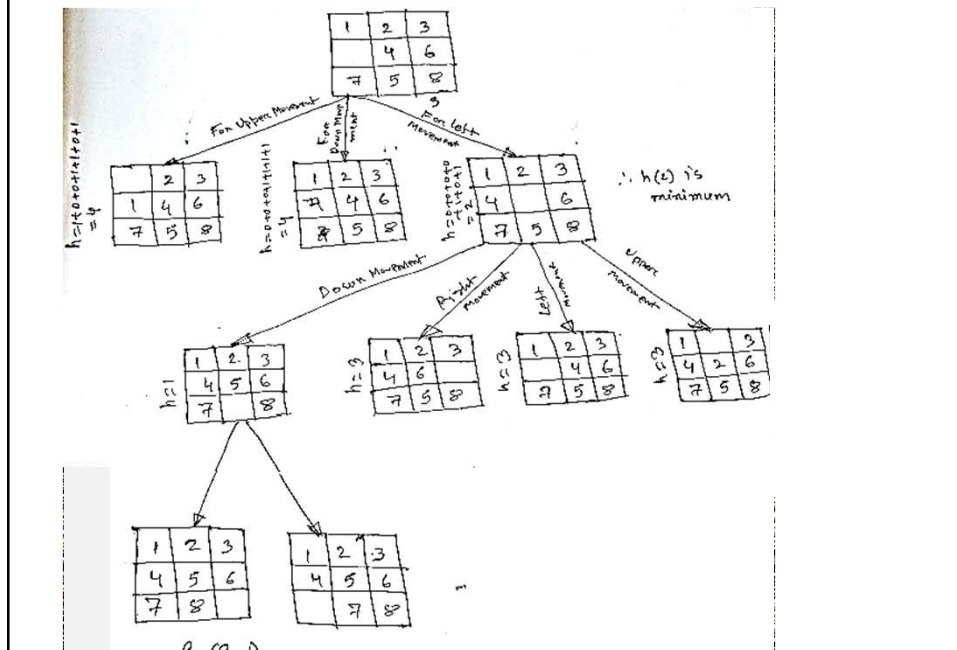
Answer: $h = \text{number of misplaced}$

$$= 0+0+0+1+1+0+0+1$$

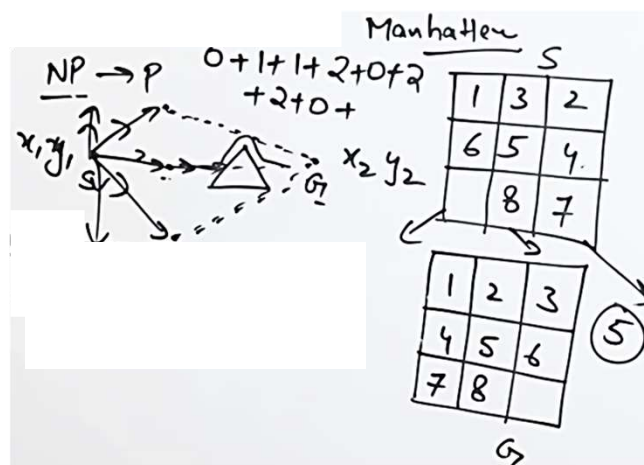
$$= 3$$

$$\therefore h(n) = h = 3$$

8-Puzzle Problem with Heuristic (Greedy Best first search)



Informed (Heuristic) Search Strategies



Problem

Solve this 8-Puzzle Problem with Heuristic

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

Greedy best-first search (Contd.)

Advantages:

- Best first search can switch between BFS and DFS by gaining the advantages of both the algorithms.
- This algorithm is more efficient than BFS and DFS algorithms.

Disadvantages:

- It can behave as an unguided depth-first search in the worst case scenario.
- It can get stuck in a loop as DFS.
- This algorithm is not optimal.

Limitation of Greedy best-first search

For this particular problem, greedy best-first search using h_{SLD} finds a solution without ever expanding a node that is not on the solution path; hence, its search cost is minimal.

(i). It is not optimal, because:

Arad - Sibiu → Fagaras → Bucharest = 140 + 99 + 211 = 450

whereas

Arad- Sibiu → Rimnicu Vilcea → Pitesti → Bucharest
= 140 + 80 + 97 + 101 = 418

i.e. the path

Sibiu → Fagaras → Bucharest

is 32 kilometers longer than the path

Sibiu → Rimnicu Vilcea → Pitesti → Bucharest

Limitation of Greedy best-first search (Contd.)

ii. Greedy best-first tree search is also incomplete even in a finite state space, much like depth-first search.

Consider the problem of getting from Iasi to Fagaras.

The heuristic suggests that Neamt be expanded first because it is closest to Fagaras, **but it is a dead end.**

The solution is

Iasi → Vaslui → Urziceni → Bucharest → Fagaras.

The algorithm will never find this solution,

because expanding Neamt puts Iasi back into the frontier,

Iasi is closer to Fagaras than Vaslui is, and so Iasi will be expanded again, leading to an infinite loop.

Greedy best-first search (Contd.)

(The graph search version is complete in finite spaces, but not in infinite ones.)

The worst-case time and space complexity for the tree version is $O(bm)$, where m is the maximum depth of the search space. With a good heuristic function, however, the complexity can be reduced substantially.

The amount of the reduction depends on the particular problem and on the quality of the heuristic.

Exercise # 1:

Prove each of the following statements, or give a counterexample: a. Breadth-first search is a special case of uniform-cost search. b. Depth-first search is a special case of best-first tree search. c. Uniform-cost search is a special case of A^* search.

Exercise # 2:

The missionaries and cannibals problem is usually stated as follows. Three missionaries and three cannibals are on one side of a river, along with a boat that can hold one or two people. Find a way to get everyone to the other side without ever leaving a group of missionaries in one place outnumbered by the cannibals in that place. This problem is famous in AI because it was the subject of the first paper that approached problem formulation from an analytical viewpoint (Amarel, 1968). a. Formulate the problem precisely, making only those distinctions necessary to ensure a valid solution. Draw a diagram of the complete state space. b. Implement and solve the problem optimally using an appropriate search algorithm. Is it a good idea to check for repeated states? c. Why do you think people have a hard time solving this puzzle, given that the state space is so simple?



THANK YOU